

# **Detecção de Defeitos em Python via Modelos de Linguagem e Verificação Formal**

---

Fernanda Alice Farias Duarte

Programa de Pós-Graduação em Informática: Disciplina de Verificação Formal

Universidade Federal do Amazonas

Manaus, 2026

# Roteiro da apresentação

---

02

| Bloco                     | O que será apresentado                                           |
|---------------------------|------------------------------------------------------------------|
| 1. Contexto e problema    | Por que Python é difícil e por que LLM/ESBMC isolados não bastam |
| 2. Objetivos e literatura | Questão de pesquisa, objetivos e lacuna                          |
| 3. Método                 | BMC, ESBMC-Python, pipeline e fluxos A/B/C                       |
| 4. Experimento            | Dataset, modelos e métricas                                      |
| 5. Resultados             | Flow C, Flow B, categorias de bug e code smells                  |
| 6. Discussão              | Descobertas, limitações, conclusão e próximos passos             |

# Contexto: por que Python é difícil de analisar?

---

03

- Python é #1 no índice TIOBE (janeiro 2026)
- É amplamente usada em web, ciência de dados e IA.
- Tipagem dinâmica: variáveis podem mudar de tipo em tempo de execução.
- Lint/mypy ajudam, mas não confirmam falhas executáveis como divisão por zero ou acesso fora dos limites

# LLMs para código: úteis, mas não suficientes

---

04

## O que fazem bem

- Identificam padrões suspeitos rápido
- Cobrem bugs executáveis e problemas de qualidade
- Servem como triagem inicial

## Onde falham

- Geram falsos positivos
- Podem alucinar operações que não existem
- Não oferecem garantia formal

**PrimeVul [Ding et al., 2025]: F1 = apenas 3,09% em vulnerabilidades reais.**

## Pontos positivos

- Confirma bugs com contraexemplos concretos.
- Quando acusa, o bug é real.
- Verifica Python diretamente.

## Limites

- Não faz triagem sozinho
- Não confirma code smells
- Custo alto: 35–50 segundos por função

**Como o ESBMC é caro (35–50s/função), ele deve ser direcionado:  
LLM aponta suspeitas; AST filtra; ESBMC confirma só bugs executáveis**

---

**Como combinar modelos de linguagem para código e verificação formal para aumentar a confiabilidade da detecção automática de defeitos em Python?**

| Objetivo | Descrição                                                                        |
|----------|----------------------------------------------------------------------------------|
| 1        | Dataset de 70 funções Python com ground truth manual validado pelo ESBMC-Python. |
| 2        | Comparar 5 modelos: 3 LLMs via API + 2 SLMs locais.                              |
| 3        | Filtrar alucinações por AST antes da verificação formal.                         |
| 4        | Integrar ESBMC-Python como confirmador formal.                                   |
| 5        | Comparar Flow A, Flow B e Flow C.                                                |
| 6        | Medir impacto via Precisão, Revocação, F1, TCF e TRN.                            |

# Trabalhos correlatos: síntese

| Linha            | Achados principais                                                                 | Lacuna                                              |
|------------------|------------------------------------------------------------------------------------|-----------------------------------------------------|
| LLMs em código   | PrimeVul: F1 = 3,09%; sensibilidade a renomeações; contexto ajuda mas não resolve. | Resultados ainda são instáveis e heurísticos.       |
| Formal em Python | ESBMC-Python verifica diretamente; EVA/PyVeritas traduzem Python → C.              | Tradução pode introduzir risco semântico.           |
| LLM + formal     | PropertyGPT e EVA combinam modelos e verificação.                                  | Pouco foco em ESBMC direto + smells.                |
| Bugs vs smells   | Categorias compartilham sinais estruturais.                                        | Modelos podem confundir bug executável e qualidade. |

Referências: [Ding et al., 2025]; [Li et al., 2026]; [Shivaji et al., 2026]; [Tihanyi et al., 2025]; [Santos et al., 2023]; [Souza et al., 2026]

**Nenhum trabalho combina, no mesmo pipeline experimental:**

- LLM + ESBMC diretamente em Python, sem tradução para C.
- Filtragem de alucinações via AST antes da verificação formal.
- Separação explícita entre bugs executáveis e code smells.
- Dataset controlado com ground truth manual e validação formal.

**Essa é a contribuição experimental do trabalho.**

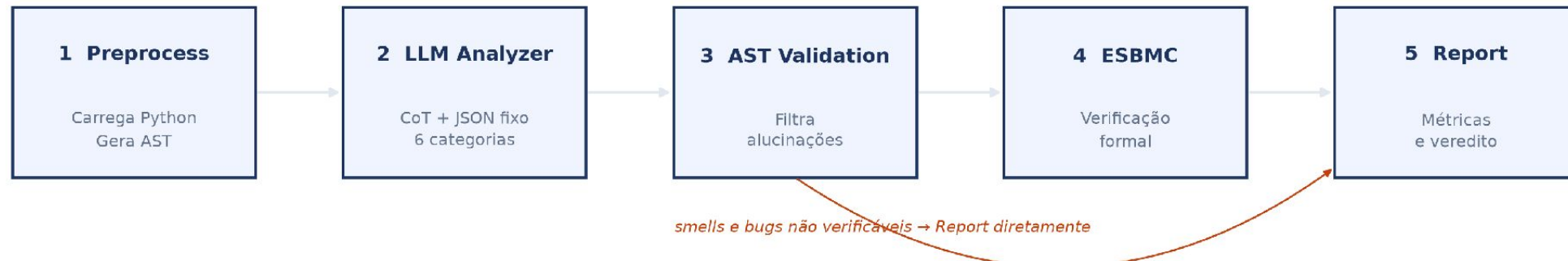
---



# Pipeline proposto: visão geral

11

## Pipeline Híbrido — LLM + Verificação Formal



**A AST não é enviada ao modelo. Ela é usada depois, internamente, para validar se a hipótese existe no código.**

# Pipeline: função de cada etapa

---

12

| <b>Etapa</b>   | <b>Função</b>                            | <b>Por que importa</b>                   |
|----------------|------------------------------------------|------------------------------------------|
| Preprocess     | Carrega código e extrai AST interna.     | Base estrutural para validação.          |
| LLM Analyzer   | Gera hipóteses em JSON fixo.             | Faz triagem e classifica bug/smell.      |
| AST Validation | Confere se a hipótese existe no código.  | Remove alucinações antes do ESBMC.       |
| ESBMC          | Verifica o arquivo original.             | Confirma VP, refuta FP ou marca timeout. |
| Report         | Consolida modelo + ESBMC + ground truth. | Calcula métricas por fluxo e categoria.  |

## Pergunta central

**“A operação citada pelo modelo existe mesmo no código?”**

## Três fases

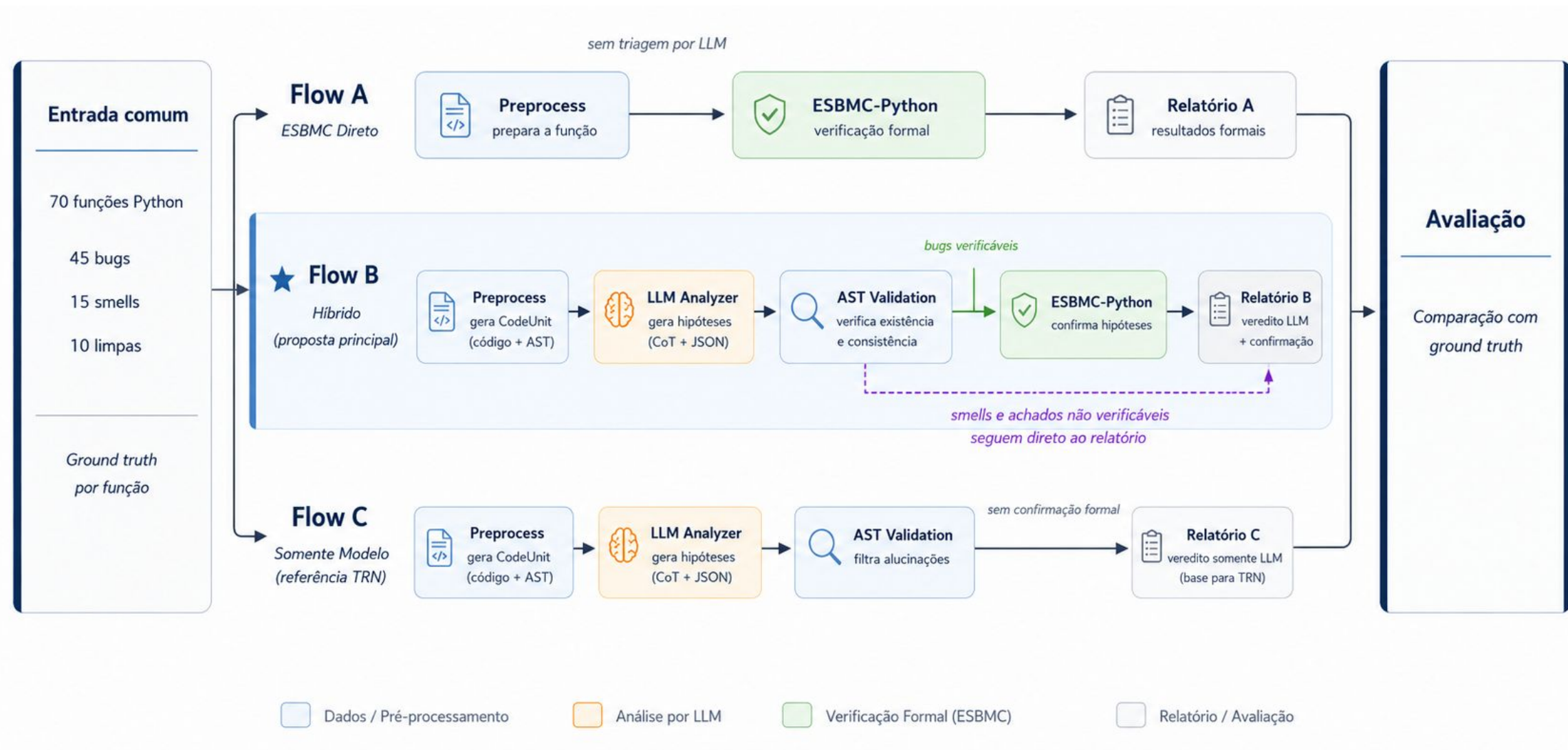
1. Correspondência direta com operações extraídas.
2. Normalização via `ast.unparse`.
3. Para `assertion_violation`, comparação com asserts presentes.

**Hipótese sem correspondência = alucinação descartada**

Evita chamar o ESBMC com código que o modelo inventou

# Os 3 fluxos de avaliação

14



Flow A = ESBMC direto · Flow B = proposta híbrida · Flow C = modelo sem confirmação formal

# Dataset: composição e ground truth

| Grupo            | Quantidade                 | Detalhe                                                        |
|------------------|----------------------------|----------------------------------------------------------------|
| Bugs executáveis | 45 funções / 48 instâncias | 15 division_by_zero; 18 out_of_bounds; 15 assertion_violation. |
| Code smells      | 15 funções                 | 5 long_method; 5 many_parameters; 5 complex_conditional.       |
| Funções limpas   | 10 funções                 | Controle para medir falsos positivos.                          |
| Validação formal | 45 arquivos de bug         | Todos confirmados com VERIFICATION FAILED.                     |

**Escopo controlado: 3 categorias de bugs verificáveis e 3 categorias de code smells.**

## LLMs via API

- gpt-5.5 - OpenAI
  - claude-sonnet-4-6 - Anthropic
  - gemini-3.1-flash-lite - Google
- Maior capacidade de raciocínio sobre código

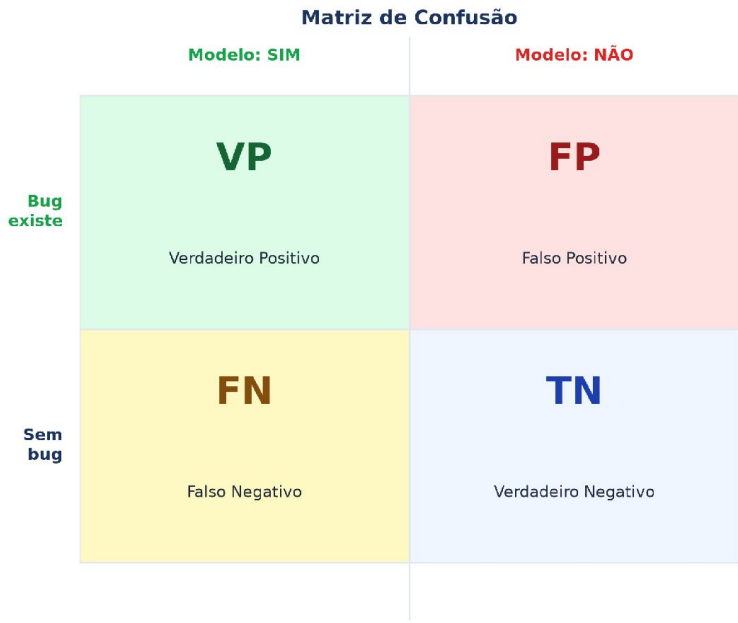
## SLMs locais via Ollama

- deepseek-r1:7b - raciocínio, 7B
- qwen2.5-coder:7b - código, 7B
- Intel Core i7-12700H · 16GB RAM · sem GPU

**Mesmo prompt para todos; temperatura = 0 quando disponível**

# Métricas: matriz de confusão

| Símbolo | Significado                             |
|---------|-----------------------------------------|
| VP      | Modelo acusou bug que realmente existe  |
| FP      | Modelo acusou bug que não existe        |
| FN      | Bug existe, mas o modelo não reportou   |
| TN      | Função limpa e modelo não reportou nada |



## Definição

### Métricas

$P = \frac{VP}{VP + FP}$

Dos alertas, q

$R = \frac{VP}{VP + FN}$

Dos bugs rea

$F1 = 2 \cdot \frac{P \cdot R}{P + R}$

Equilibr

$TCF = \frac{P + R}{2}$

Hipóteses confi

$TRN = \frac{FP}{FP + FN}$

Proporção d

$MCC \in [-1, +1]$

# Métricas: como interpretar P, R e F1

---

18

Precisão

**P**

Dos alertas emitidos, quantos eram bugs reais?

Revocação

**R**

Dos bugs reais, quantos o modelo encontrou?

F1-score

**F1**

Resumo entre precisão e revocação.

**Se o ESBMC remove FPs, a precisão sobe. Se o modelo já perdeu bugs, a revocação não sobe.**



TCF

## confirmação

Das hipóteses enviadas ao ESBMC, quantas foram confirmadas?

TRN

## redução de ruído

Dos FPs do modelo, quantos o ESBMC eliminou?

- $TRN = 1,000$  significa que o ESBMC eliminou 100% dos falsos positivos
- TCF alta significa que o modelo levantou boas hipóteses verificáveis
- Essas métricas separam qualidade da triagem e poder de confirmação formal

# Flow C: contagens principais

Flow C = modelo + AST, sem ESBMC. Aqui vemos o que o modelo fez sozinho.

| Modelo                | VP | FP | FN | Aluc. |
|-----------------------|----|----|----|-------|
| gpt-5.5               | 48 | 0  | 0  | 0     |
| claude-sonnet-4-6     | 48 | 4  | 0  | 1     |
| gemini-3.1-flash-lite | 46 | 3  | 2  | 1     |
| deepseek-r1:7b        | 27 | 8  | 21 | 7     |
| qwen2.5-coder:7b      | 34 | 20 | 14 | 13    |

**Leitura rápida: FP = alarme falso; FN = bug perdido. DeepSeek perdeu 21 bugs; Qwen gerou 20 alarmes falsos**

Legenda: VP = bugs encontrados · FP = alarmes falsos · FN = bugs perdidos · Aluc. = hipótese inventada pelo modelo

# Flow C: métricas principais

21

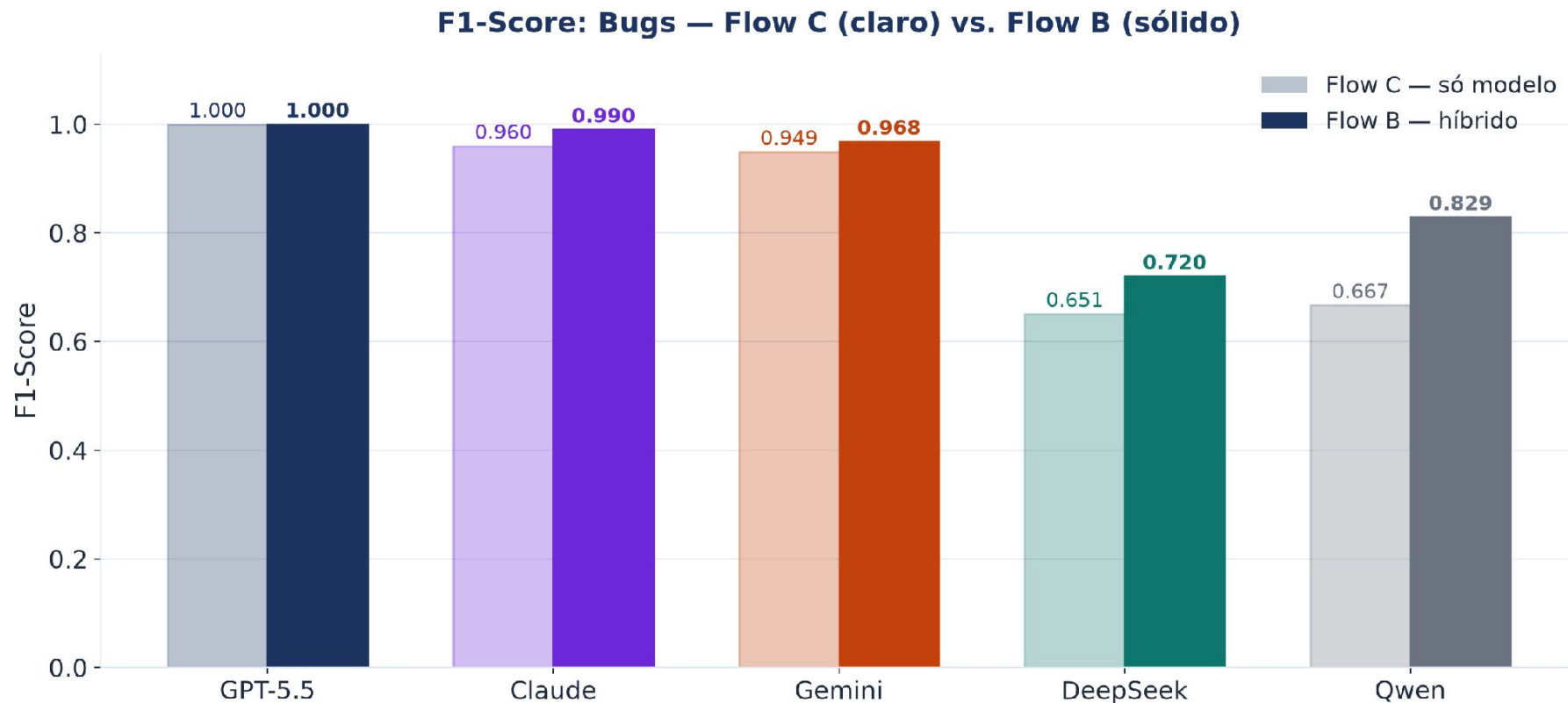
Legenda: P = precisão · R = revocação · F1 = equilíbrio entre P e R · MCC = qualidade por função

| Modelo   | P     | R     | F1    | MCC    |
|----------|-------|-------|-------|--------|
| gpt-5.5  | 1,000 | 1,000 | 1,000 | 1,000  |
| claude   | 0,923 | 1,000 | 0,960 | 0,938  |
| gemini   | 0,939 | 0,958 | 0,949 | 0,825  |
| deepseek | 0,771 | 0,563 | 0,651 | 0,386  |
| qwen     | 0,630 | 0,708 | 0,667 | -0,059 |

- GPT: P=R=F1=1,000, sem perda e sem falso positivo
- SLMs caem por motivos diferentes: DeepSeek perde bugs; Qwen gera ruído

# Flow C:

- O gráfico compara o F1 antes e depois do ESBMC.
- No Flow C, o modelo ainda não passou pelo verificador formal.
- Por isso, FPs ainda contam contra a precisão.
- FNs já são um problema estrutural: se o modelo não apontou o bug, o ESBMC não terá o que confirmar depois.



# Flow B: contagens principais

Flow B = modelo + AST + ESBMC. Aqui o verificador remove suspeitas falsas.

| Modelo                | VP | FP | FN | TRN   |
|-----------------------|----|----|----|-------|
| gpt-5.5               | 48 | 0  | 0  | ---   |
| claude-sonnet-4-6     | 48 | 1  | 0  | 0,750 |
| gemini-3.1-flash-lite | 46 | 1  | 2  | 0,667 |
| deepseek-r1:7b        | 27 | 0  | 21 | 1,000 |
| qwen2.5-coder:7b      | 34 | 0  | 14 | 1,000 |

Leitura rápida: o ESBMC derrubou FPs, mas os FNs continuam iguais.

Legenda: VP = bugs confirmados · FP = suspeitas falsas restantes · FN = bugs perdidos · TRN = FPs eliminados pelo ESBMC

# Flow B: métricas principais

Legenda: P = precisão · R = revocação · F1 = equilíbrio · TCF = hipóteses confirmadas pelo ESBMC

| Modelo   | P     | R     | F1    | TCF   |
|----------|-------|-------|-------|-------|
| gpt-5.5  | 1,000 | 1,000 | 1,000 | 1,000 |
| claude   | 0,980 | 1,000 | 0,990 | 0,960 |
| gemini   | 0,979 | 0,958 | 0,968 | 0,958 |
| deepseek | 1,000 | 0,563 | 0,720 | 0,964 |
| qwen     | 1,000 | 0,708 | 0,829 | 0,850 |

- O ESBMC aumenta precisão ao remover FPs.
- A revocação não sobe: bugs não apontados pelo modelo continuam como FN.

# Flow B: o que o ESBMC corrigiu

25

Claude: reduziu os falsos positivos de 4 para 1, elevando o F1 de 0,960 para 0,990.

Gemini: reduziu os falsos positivos de 3 para 1, com F1 passando de 0,949 para 0,968.

DeepSeek: eliminou os 8 falsos positivos, aumentando a precisão de 0,771 para 1,000.

Qwen: eliminou os 20 falsos positivos, aumentando a precisão de 0,630 para 1,000.

- O ESBMC é árbitro, não detector.
- Ele confirma ou refuta o que o modelo já apontou.
- Se o modelo não apontou o bug, o bug continua perdido.

## Redução de Falsos Positivos: Flow C → Flow B



# Resultado central: FP cai, FN permanece

---

26

| Modelo   | FP Flow C | FP Flow B | FN Flow C | FN Flow B |
|----------|-----------|-----------|-----------|-----------|
| Claude   | 4         | 1         | 0         | 0         |
| Gemini   | 3         | 1         | 2         | 2         |
| DeepSeek | 8         | 0         | 21        | 21        |
| Qwen     | 20        | 0         | 14        | 14        |

**O híbrido resolve ruído, não cobertura**

---

Legenda: FP Flow C/B = alarmes falsos antes/depois do ESBMC · FN Flow C/B = bugs perdidos antes/depois do ESBMC



# Categorias: assertion e division

## ASSERTION\_VIOLATION

| Modelo   | VP | FP | F1    |
|----------|----|----|-------|
| gpt      | 15 | 0  | 1,000 |
| claude   | 15 | 0  | 1,000 |
| gemini   | 13 | 0  | 0,929 |
| deepseek | 11 | 0  | 0,846 |
| qwen     | 14 | 0  | 0,966 |

Mais fácil: o assert aparece explicitamente no código.

## DIVISION\_BY\_ZERO

| Modelo   | VP | FP | F1    |
|----------|----|----|-------|
| gpt      | 15 | 0  | 1,000 |
| claude   | 15 | 1  | 0,968 |
| gemini   | 15 | 1  | 0,968 |
| deepseek | 9  | 0  | 0,750 |
| qwen     | 15 | 0  | 1,000 |

ESBMC removeu FPs; qwen chegou a F1=1,000.

# Categoria mais difícil: out\_of\_bounds

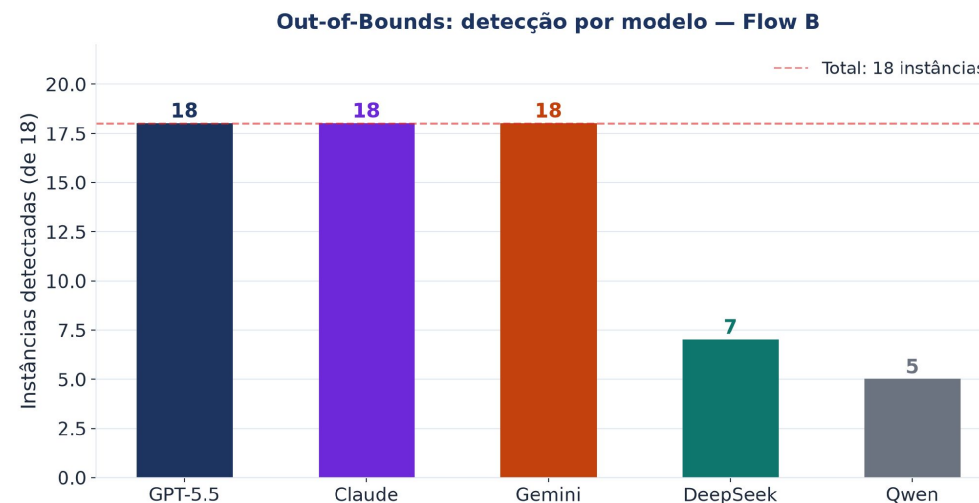
28

| Modelo   | VP | FP | F1    |
|----------|----|----|-------|
| gpt-5.5  | 18 | 0  | 1,000 |
| claude   | 18 | 0  | 1,000 |
| gemini   | 18 | 0  | 1,000 |
| deepseek | 7  | 0  | 0,560 |
| qwen     | 5  | 0  | 0,435 |

## Por que é difícil?

- Exige saber quais índices são usados.
- Exige inferir o tamanho real da lista.
- Exige verificar se existe caminho onde índice  $\geq$  tamanho.

Legenda: VP = out\_of\_bounds encontrados · FP = alarmes falsos · F1 = desempenho geral na categoria



# Code smells: tabela geral

Flow B = Flow C para smells, porque ESBMC não se aplica a problemas de qualidade.

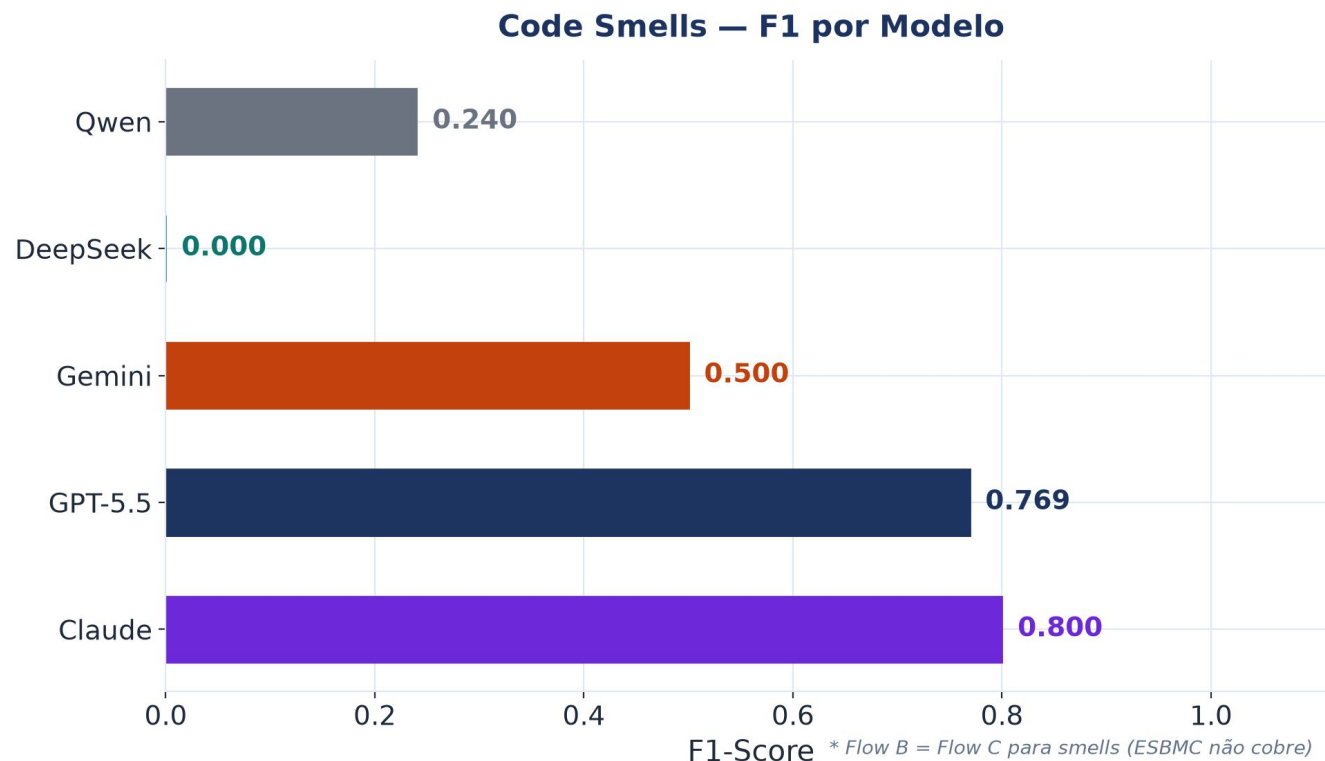
| Modelo                | VP | FP | FN | P     | R     | F1    |
|-----------------------|----|----|----|-------|-------|-------|
| claude-sonnet-4-6     | 10 | 0  | 5  | 1,000 | 0,667 | 0,800 |
| gpt-5.5               | 10 | 1  | 5  | 0,909 | 0,667 | 0,769 |
| gemini-3.1-flash-lite | 5  | 0  | 10 | 1,000 | 0,333 | 0,500 |
| deepseek-r1:7b        | 0  | 0  | 15 | 0,000 | 0,000 | 0,000 |
| qwen2.5-coder:7b      | 15 | 95 | 0  | 0,136 | 1,000 | 0,240 |

Legenda: VP encontrados · FP falsos alarmes · FN perdidos · P/R/F1 = desempenho

# Code smells: como interpretar

30

- Claude foi o melhor em smells (F1=0,800); GPT ficou próximo (0,769)
- Gemini e DeepSeek foram conservadores: poucos/nenhum FP, mas muitos FNs
- Qwen achou todos os smells reais, mas gerou 95 FPs (P=0,136)
- many\_parameters é fácil porque basta contar parâmetros.
- long\_method foi zero para todos; precisa de filtro próprio.



# Discussão: 4 descobertas centrais

| RQ  | Resposta                         | Evidência                                 |
|-----|----------------------------------|-------------------------------------------|
| RQ1 | Sim. ESBMC reduz FPs.            | Qwen FP 20→0; DeepSeek FP 8→0.            |
| RQ2 | Sim. AST separa alucinações.     | GPT=0; DeepSeek=7; Qwen=13.               |
| RQ3 | Não. ESBMC não recupera FNs.     | DeepSeek manteve 21 FNs; Qwen manteve 14. |
| RQ4 | Não. Smells seguem outra lógica. | Qwen teve 95 FPs; long_method teve F1=0.  |

Legenda: RQ = questão de pesquisa · Evidência = número principal que sustenta a resposta

| Eixo        | Limitação                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| Dataset     | 70 funções isoladas e controladas; criadas manualmente; sem bootstrap nesta versão.                               |
| Pipeline    | Só verifica hipóteses levantadas pelo modelo; cobertura limitada pela triagem; 3 casos não confirmados no limite. |
| Modelos     | SLMs em CPU sem GPU; latência alta; temperatura não configurável em GPT-5.5 e Claude.                             |
| Code smells | Sem solução formal para FPs; critérios objetivos não capturam aspectos subjetivos.                                |

- Confirmação formal reduziu FPs em todos os modelos
- gpt-5.5: F1=1,000 mesmo sem ESBMC
- Claude Flow B: F1=0,990
- Qwen teve maior ganho de precisão: 0,630→1,000
- ESBMC não recupera FN
- long\_method exige estratégia própria.

# Próximos passos e perguntas

---

- Dataset V2: mais funções, mais categorias e funções extraídas de repositórios reais
- Avaliação estatística: bootstrap para medir significância
- Filtro específico para code smells, especialmente long\_method
- Explorar contraexemplos do ESBMC para validar comportamento Python original
- Prompts específicos por categoria, como raciocínio sobre índices para out\_of\_bounds